

# Code-Layout, Readability and Reusability

|               |                      |
|---------------|----------------------|
| Date          | 19 September 2026    |
| Team ID       | SWTID-2026-3976      |
| Project Name  | AI FAQ Assistant API |
| Maximum Marks | 2 Marks              |

## Code Layout, Readability and Reusability

The AI FAQ Assistant follows a modular layout under `src/`, keeping every concern in its own folder so a new route or AI feature can be added without touching unrelated files.

| Folder                    | Purpose                                                                                                                                                                       | Reusability Practice                                                                                                                                                                                                          |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| config/db.js              | MongoDB connection setup                                                                                                                                                      | Connection logic centralized once, invoked from <code>server.js</code> before the Express listener starts                                                                                                                     |
| controllers/              | Request-handling logic per resource ( <code>authController.js</code> , <code>faqController.js</code> , <code>aiController.js</code> )                                         | Thin controllers delegate persistence to Models and every Gemini call to <code>services/geminiService.js</code> , so AI-vendor logic isn't duplicated                                                                         |
| middleware/               | <code>authMiddleware.js</code> (JWT protect guard), <code>validationMiddleware.js</code> (request validation), <code>errorMiddleware.js</code> (centralized error formatting) | A single <code>protect</code> guard is reused across every protected route in <code>faqRoutes.js</code> and <code>aiRoutes.js</code> instead of re-checking tokens per controller                                             |
| models/                   | Mongoose schemas ( <code>User.js</code> , <code>FAQ.js</code> ) with validation baked in                                                                                      | Required-field, unique-email and category-enum validation lives once in the schema, not repeated in every controller                                                                                                          |
| routes/                   | Endpoint-to-controller mapping per resource ( <code>authRoutes.js</code> , <code>faqRoutes.js</code> , <code>aiRoutes.js</code> )                                             | Each route file is mounted with its own prefix ( <code>/api/auth</code> , <code>/api/faqs</code> , <code>/api/ai</code> ); the FAQ search route is declared before <code>/:id</code> specifically to avoid a routing conflict |
| services/geminiService.js | Wraps the Google Gemini SDK behind reusable functions for direct answers and FAQ-pair generation                                                                              | Both AI features call the same service module with a different prompt shape, avoiding duplicated Gemini client setup                                                                                                          |

Consistent naming (*camelCase* functions, *PascalCase* models), small single-purpose controller functions, and comments above non-obvious logic (e.g. why the search route is registered before `/:id` , and how `errorMiddleware.js` respects a status code already set by the Gemini SDK) keep the codebase readable for all 5 contributors working in parallel.